

Patrón de diseño MVC en Laravel

*Modelos, Vistas y Controladores
(además de rutas)*

Proyecto de finanzas personales

- **Objetivo:** aprender los conceptos fundamentales de Laravel mediante la creación de un pequeño sistema de gestión de finanzas personales/familiares.
- **Desarrollo:** durante las clases de la presente materia.
- **Sugerencias:**
 - Seguir el desarrollo del código a la par del profesor
 - Tener al día la materia; si falta es mi responsabilidad ponerme al día
 - Preguntar todo lo que haga falta en clases

Descripción

Necesitamos un sistema web que nos permita gestionar las finanzas personales (y opcionalmente los de nuestra familia). El sistema deberá estar protegido mediante una sesión de usuario, por lo cual es fundamental contar con un sistema de registro y de inicio de sesión. Cada ingreso o egreso estará categorizado para facilitar el análisis de los grupos de transacciones; también podrán existir transacciones compartidas por grupos de personas. Además, el sistema deberá permitir la elaboración de informes de ingresos y gastos, los cuales podrán ser filtrados mediante distintos criterios, como pueden ser las categorías, usuarios o rangos de fechas.

Características del sistema

- **Autenticación de Usuarios (Personalizada):**
 - Registro de nuevos usuarios (nombre, email, contraseña).
 - Inicio de sesión de usuarios existentes.
 - Cierre de sesión.
 - Posibilidad de recuperación de contraseña (opcional).
- **Gestión de Usuarios, Roles y Permisos:**
 - Definición de diferentes roles de usuario (ej.: "Administrador", "Usuario").
 - Asignación de roles a los usuarios.
 - Definición de permisos específicos (ej.: "crear transacciones", "editar categorías", "ver informes").
 - Control de acceso a ciertas funcionalidades basado en roles y permisos.

Características del sistema

- **Gestión de Transacciones:**

- Creación de nuevos ingresos (descripción, monto, fecha, categoría).
- Creación de nuevos gastos (descripción, monto, fecha, categoría).
- Listado de todas las transacciones del usuario (ordenadas por fecha).
- Edición de transacciones existentes.
- Eliminación de transacciones.

- **Gestión de Categorías:**

- Visualización de las categorías de ingresos y gastos.
- Creación de nuevas categorías (tipo: ingreso/gasto).
- Edición de categorías existentes.
- Eliminación de categorías (validando que no estén en uso).

Características del sistema

- **Gestión de Grupos Compartidos:**

- Creación de grupos (ej: "Familia Pérez").
- Invitación de usuarios a un grupo.
- Visualización de los miembros de un grupo.

- **Transacciones Compartidas:**

- Asociación de transacciones a un grupo compartido (además de al usuario individual).
- Visualización de las transacciones de un grupo al que pertenece el usuario.
- Posibilidad de filtrar transacciones por usuario individual o por grupo compartido.

Características del sistema

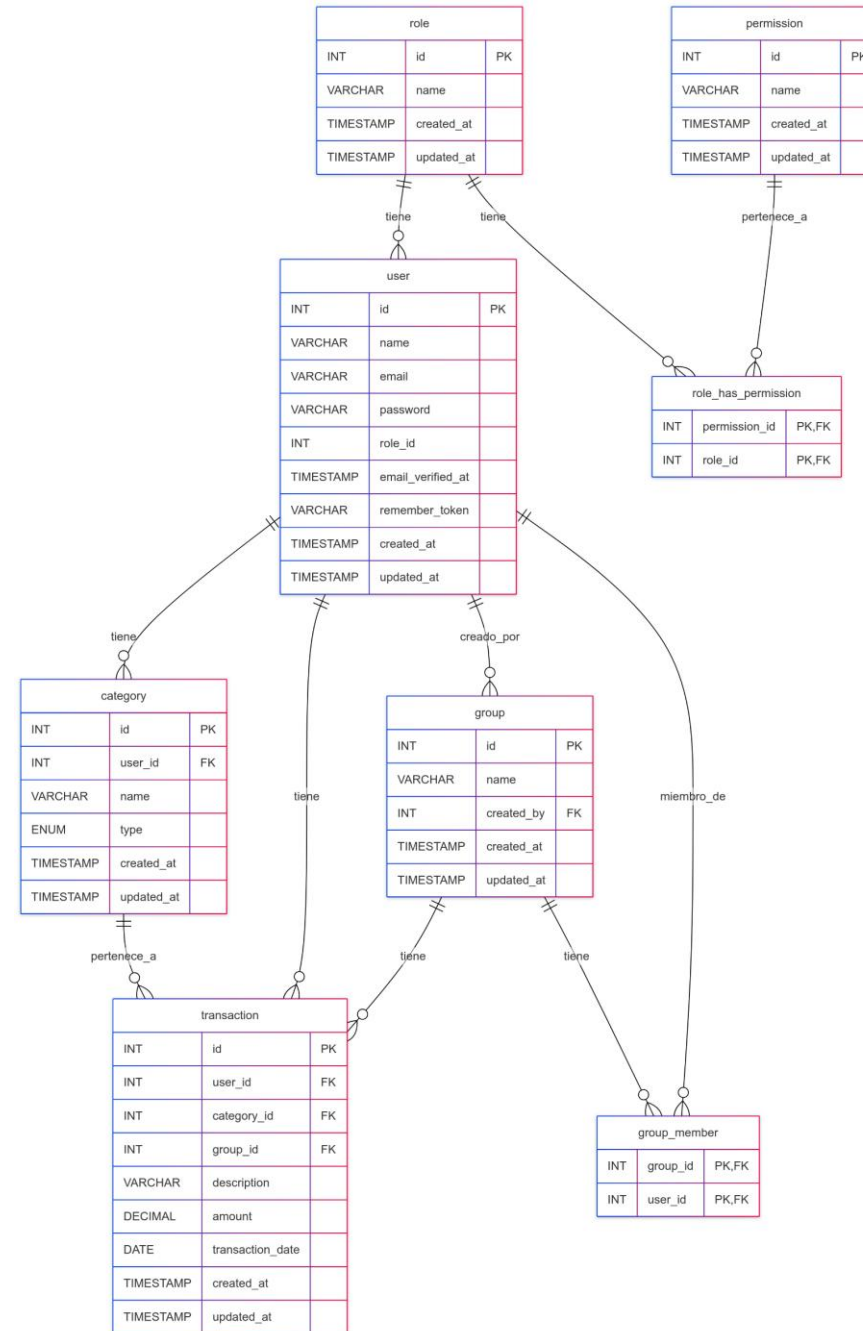
- **Informes y Resúmenes:**

- Resumen de ingresos totales (individual y por grupo) por período.
- Resumen de gastos totales (individual y por grupo) por período.
- Visualización del saldo (individual y por grupo) por período.
- Gráfico de distribución de gastos por categoría (individual y por grupo) en un período.
- Filtrado de transacciones por categoría, rango de fechas y por usuario/grupo.

Modelo de datos

Tablas:

- Usuarios
- Roles
- Permisos
- Roles_permisos
- Transacciones
- Categorías
- Grupos
- Miembro_grupo



Componentes/conceptos a usar

- **Rutas:** definición de rutas comunes y rutas de administración, grupos de rutas y vinculación con controladores REST
- **Controladores:** definición de controladores REST
- **Modelos:** creación de modelos para obtención de datos de la base de datos MySQL
- **Vistas:** se crearán vistas para cada características funcional del sistema, incluyendo formularios
- **Blade:** creación de componentes visuales para las vistas
- **Migraciones:** definición de la estructura de las tablas
- **Seeders:** definición de datos de prueba para las tablas
- **Middlewares:** restricción de recursos mediante autenticación y autorización + roles y permisos

Componentes/conceptos a usar

- **Validación:** creación de FormRequest para validar datos ingresados mediante un formulario
- **Autenticación:** lógica de registro, inicio de sesión y cierre de sesión
- **Mailing:** uso de un servidor SMTP para enviar correos
- **Notificaciones:** visualizar notificaciones en el sistema
- **Testing:** comprobar que el sistema funciona de la forma correcta
- **Observabilidad (opcional):** visualizar errores en tiempo real
- **Informes:** visualización de informes mediante gráficos y reportes en formato PDF (opcional)

Conceptos de seguridad

- Nociones de protección de credenciales sensibles
- Protección contra ataques CSRF (Cross-Site Request Forgery)
- Prevención de XSS (Cross-Site Scripting)
- Protección contra inyección SQL
- Almacenamiento seguro de contraseñas (hashing)
- Validación y saneamiento de datos de formularios
- Autenticación de usuarios
- Autorización de usuarios

Patrón MVC en Laravel

1. En el archivo *routes/web.php* se define la ruta que el usuario visita. Esa ruta apunta a un controlador específico.
2. Se busca el controlador en la carpeta *app/Http/Controllers* y se ejecuta el método definido en la correspondiente ruta.
3. Si lo necesita, el controlador interactúa con uno o más modelos (ubicados en la carpeta *app/Models*). Como resultado, un controlador puede devolver una vista junto con los datos procesados.
4. Las vistas se ubican en la carpeta *resources/views*. En ellas se maqueta la página junto con los datos recibidos.

Ruta

- Definición de una ruta que apunta a un controlador:

```
Route::get('/transacciones', [TransaccionesController::class,  
    'index'])  
    ->name('transacciones.index');
```

- *Route::get* define una URL que podemos visitar con el navegador
- *'/transacciones'* es el fragmento que se adiciona al dominio del sitio para generar la URL completa. Ejemplo: <https://misitio.com/transacciones>
- *TransaccionesController::class* es el controlador al cual apunta la ruta, e *'index'* es el método del controlador que se ejecutará
- *name('transacciones.index')* define el nombre de la ruta, el cual usaremos en otras partes del proyecto

Controlador

- Creación de un controlador:

```
php artisan make:controller TransaccionesController
```

- Estructura:

```
class TransaccionesController extends Controller {  
    public function index() {  
        return view('transacciones', ['datos' => $datos]);  
    }  
}
```

- *view()* es una función que automáticamente devuelve una vista, el primer argumento es el nombre de la vista y el segundo es un arreglo con los datos que se enviarán.

Modelo

- Como aun no tenemos conexión con la base de datos, crearemos una clase con un arreglo de datos para consumir con el controlador

```
public static function getDatos(): array {  
    $transacciones = [  
        [  
            'user_id' => 1,  
            'description' => 'Compra de supermercado',  
            'amount' => 50000.75,  
            'transaction_date' => '2025-01-15',  
            'created_at' => $now,  
            'updated_at' => $now,  
        ],  
    ],  
    ...  
}
```

Obtener los datos en el controlador

- Ahora podemos leer los datos de la transacción en el controlador de la siguiente forma:

```
public function index() {  
    $datos = Transacciones::getDatos();  
    return view('transacciones', ['datos' => $datos]);  
}
```

- En la vista tendremos a disposición una variable *\$datos* con la información de las transacciones (**importante**: la variable se llamará como nosotros definamos su clave en el arreglo de datos)

Vista

- Creamos la vista de forma manual en la carpeta *resources/views* con el nombre *transacciones.blade.php*.
- *.blade* es una extensión que indica que este archivo utiliza el sistema de plantillas **Blade**, el cual estudiaremos próximamente.
- Al ser una vista, aquí podemos ingresar todo el código HTML, CSS y Javascript que conocemos, y también código PHP, lo cual será útil para analizar los datos provistos desde el controlador.

Ejemplo de vista

```
<h1>Listado de Transacciones</h1>
<!-- Asumiendo que estamos creando una tabla -->
<?php foreach ($transacciones as $transaccion): ?>
    <tr>
        <td><?php echo $transaccion['user_id']; ?></td>
        <td><?php echo $transaccion['description']; ?></td>
        <td><?php echo $transaccion['amount']; ?></td>
        <td><?php echo $transaccion['transaction_date']; ?></td>
        <td><?php echo $transaccion['created_at']; ?></td>
        <td><?php echo $transaccion['updated_at']; ?></td>
    </tr>
<?php endforeach; ?>
```

Recapitulando – Pasos a seguir

1. Definir la ruta que deseamos crear, incluyendo el tipo de método a usar (GET para enlaces, POST para formularios de subida de información), el controlador y el método al cual apuntar (no olvidarse de ponerle nombre a la ruta).
2. Crear el controlador y el método a ser llamado por la ruta. Incluir el código necesario para procesar toda la información necesaria, como el llamado a modelos o clases utilitarias. Por último, llamar a la vista que recibirá los datos procesados.
3. Crear la vista, utilizando HTML, CSS y Javascript, y mostrar los datos recibidos desde el controlador.